"Life is Good" - IGR PROJECT

KAREN EL KHOURY, Institut Polytechnique de Paris, France

1 INTRODUCTION

My project *Life Is Good* represents a cute scene in a flower field where we have a minecraft bee flying around, a minecraft villager walking around in this garden with an apple tree, and a rhino. The apple falls from the tree, and when the lights turn into dico mode, the villager starts dancing! Life is really good sometimes if we know how to enjoy it like this villager! The part i tried to focus on the most was the simulation , where i tried to implement 3D SPH in my scene. In this report, i will first show my results. In the second part i will do a Litterature survey about techniques used in different fields for fluid simulation. Then i will go into technical details: For Geometry : Laplacian Smoothing, Taubian Smoothing, and Mesh Decimation. For Rendering : Shadow Mapping and RayTracing. For Simulation : Rigid Body falling (apple) and 3D SPH.

2 RESULTS

Fig 1 shows the rendered scene without the sph componenet. As mentioned, i have a low poly tree .obj mesh, 2 minecraft .obj meshes that i got from Blockbench. The floor is a simple plane tiled with a small flower pattern that i created myself using paint, then we repeat the pattern to tile the whole plane. And i have 2 .off meshes : The rhino and the apple. Fig 2 shows my tentative implementation of 3D SPH with particles represented by small blue spheres. Fig 3 pictures the disco mode of my scene: Natural lights are turned off, and the disco lights (multicolored lights, in this frame the lights are pink) are turned on. The villager then starts dancing and bouncing around. Fig 8 shows the animation i was able to generate by puting next to each other the ray traced result. We can see the apple falling and then bouncing on the floor.

3 RELATED WORK

My literature survey focuses on understanding which fluid simulation techniques are still commonly used today and why they remain relevant. Rather than attempting to cover all recent advances, the goal is to identify classical methods that continue to serve as the foundation of modern computer graphics pipelines.

Many of the techniques currently used in visual effects, games, and interactive applications are based on algorithms introduced several decades ago. Approaches such as particle–grid methods, Smoothed Particle Hydrodynamics (SPH), and real-time approximations are still widely employed, either in their original form or through modern adaptations. Studying these methods provides essential insight into how contemporary systems balance physical accuracy, performance constraints, and visual quality.

The sources used in this survey include influential research papers that are still referenced and applied today, as well as more accessible resources such as online lecture notes, technical blogs„.

Fluid simulation has long been a central research topic in computer graphics, with applications ranging from cinematic visual effects to real-time interactive systems and, more recently, machine-learning–assisted simulation frameworks. Although all approaches are derived from the incompressible Navier–Stokes equations, their algorithmic realizations vary due to constraints on computational cost, numerical stability, and interactivity.

Author's address: Karen EL KHOURY, Institut Polytechnique de Paris, France.

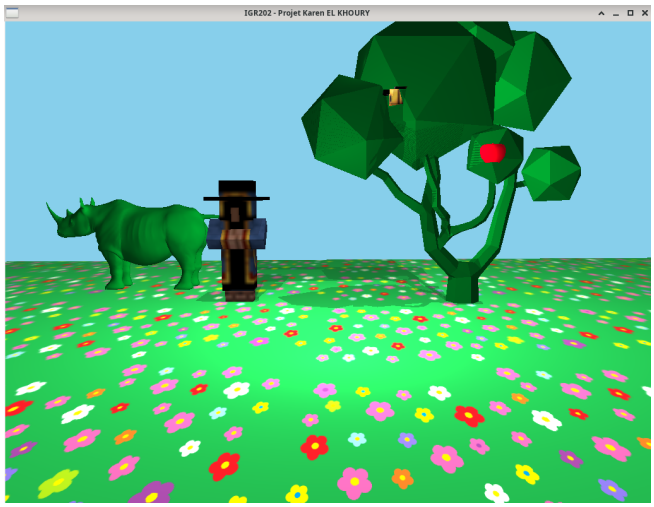

Fig. 1. "Life is Good" Rendered Scene

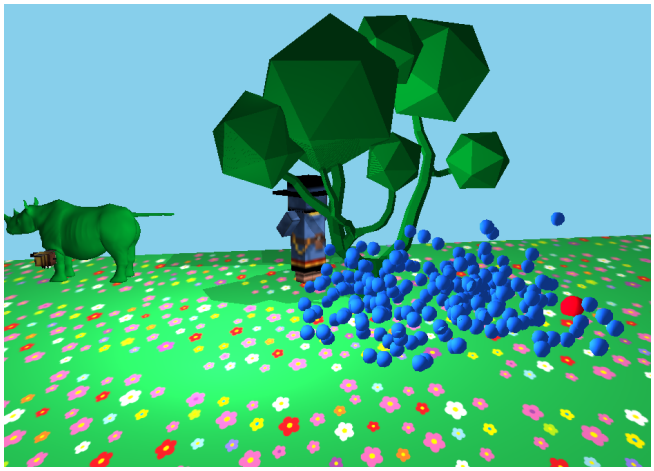

Fig. 2. Scene with the 3d SPH component rendered

3.1 Offline Fluid Simulation

Offline fluid simulation methods prioritize physical accuracy and visual realism over computational performance. In production pipelines for film and high-end visual effects, simulation times of several minutes to hours per frame are acceptable, allowing the use of

Fig. 3. Villager bouncing and dancing as the disco lights turns on

high-resolution discretizations, global numerical solvers, and complex surface representations. As a result, these methods remain the industry standard for high-quality liquid simulation.

3.1.1 Hybrid Particle–Grid Methods. The dominant approach for offline liquid simulation is the family of hybrid particle–grid methods, commonly referred to as the Particle-in-Cell (PIC) family. In particular, FLIP (Fluid–Implicit Particle) and its extensions are widely used in production. These methods combine two complementary representations: Lagrangian particles are used for advection and fine-scale detail preservation, while an Eulerian grid is employed to enforce incompressibility and solve pressure.

At a theoretical level, fluid motion is governed by the incompressible Navier–Stokes equations. In FLIP-based solvers, incompressibility is enforced through a pressure projection step performed on the grid.

3.1.2 FLIP Algorithm. A typical FLIP simulation step consists of the following stages:

(1) Transfer particle velocity and mass to the grid.
(2) Apply grid-based forces such as gravity and viscosity.
(3) Perform pressure projection by solving a global Poisson equation to enforce a divergence-free velocity field.
(4) Transfer velocity updates from the grid back to the particles, where particles receive only velocity increments.
(5) Advect particles using their updated velocities.

The pressure projection step is the key component of the algorithm and typically solves a Poisson equation of the form:

$$\nabla^2 p = \frac{1}{\Delta t} \nabla \cdot \mathbf{u},$$

ensuring incompressibility of the fluid.

3.1.3 APIC and Extensions. Several extensions to FLIP have been proposed to improve numerical behavior and visual quality. One of the most influential is APIC (Affine Particle–in–Cell), which augments each particle with a local affine velocity field. This extension improves angular momentum conservation and better captures rotational flow behavior while reducing numerical dissipation. Due to these advantages, APIC has been widely adopted in modern production solvers.

3.1.4 Surface Representation and Reconstruction. Accurate surface representation is essential for rendering high-quality liquids. In offline pipelines, fluids are typically represented implicitly using level sets or signed distance fields. These representations allow robust handling of complex topological changes such as splashes, merging flows, and breaking sheets. For rendering, explicit surface meshes are reconstructed using algorithms such as Marching Cubes or Dual Contouring.

3.1.5 Limitations of Offline Methods. Despite their accuracy and visual fidelity, offline fluid simulation methods have several limitations. They are computationally expensive, rely on global linear solvers, and require careful parameter tuning. As a result, these methods are impractical for real-time interaction and are primarily suited to offline production environments.

3.2 Real-Time Fluid Simulation

Real-time fluid simulation targets interactive applications operating under strict frame-time constraints, typically 16–33 ms per frame. In this context, solver design prioritizes stability, predictability, and computational efficiency over strict physical accuracy. As a result, real-time methods often rely on simplified formulations and approximations that provide visually plausible results at interactive rates.

3.2.1 Position-Based Fluids (PBF). One of the most widely used real-time particle-based approaches is Position-Based Fluids (PBF). PBF reformulates fluid simulation as a constraint satisfaction problem rather than a force-based integration scheme. Instead of computing forces and integrating accelerations, PBF directly enforces constraints on particle positions, most notably density constraints that approximate incompressibility.

At each time step, particle positions are predicted and then iteratively corrected to satisfy these constraints. This approach provides excellent numerical stability and allows relatively large time steps, making it well suited for real-time applications. However, incompressibility is only approximated, and the method is less physically faithful than Navier–Stokes–based solvers. For this reason, PBF is primarily used in games and interactive applications where robustness and performance are more important than physical accuracy.

3.2.2 Smoothed Particle Hydrodynamics (SPH). While PBF prioritizes stability and simplicity, Smoothed Particle Hydrodynamics (SPH) provides a more physically grounded particle-based formulation for real-time and interactive fluid simulation. SPH represents fluids as a set of particles that move with the flow and interact through kernel-based force computations. This Lagrangian formulation naturally handles large deformations such as splashes, breaking waves, and free-surface motion without mesh distortion.

SPH is also well suited for fluid–solid coupling, as interactions with rigid bodies can be expressed directly through particle forces. Moreover, SPH relies on local neighborhood interactions, making it highly parallelizable and suitable for GPU and multi-core implementations. As a result, SPH occupies a middle ground between purely position-based methods and more expensive grid-based solvers, offering improved physical behavior while remaining compatible with real-time constraints.

3.2.3 Baseline SPH Algorithm. Most SPH solvers follow a similar high-level structure. A standard weakly compressible SPH (WCSPH) time step consists of the following stages:

(1) Neighbor search using a spatial acceleration structure.
(2) Density estimation by summing kernel contributions from neighboring particles.
(3) Pressure computation using an equation of state.
(4) Force computation, including pressure forces, viscosity, and external forces such as gravity.
(5) Time integration of particle velocities and positions.
(6) Boundary handling and collision response.

This formulation is conceptually simple and easy to implement but is numerically stiff and often requires very small time steps to maintain stability.

3.2.4 SPH and PBF in Practice. In practice, PBF and SPH are used for different purposes. PBF is favored in games and highly interactive environments due to its robustness and simplicity, while SPH is employed when improved physical realism and fluid–solid interaction are required. Both methods are widely used in production tools such as Autodesk Bifrost, Houdini fluid solvers, and Blender's Mantaflow, as well as in engineering applications including dam-break simulations and granular–fluid coupling.

3.3 Machine Learning and Hybrid Approaches

In recent years, machine learning has increasingly been explored in the context of fluid simulation. However, rather than replacing classical physics-based solvers, machine learning is primarily used to augment them. Most modern approaches rely on hybrid physics–ML frameworks, where learning-based components accelerate or approximate specific parts of the simulation pipeline while classical solvers ensure physical correctness and stability.

3.3.1 Motivation and Scope. A major motivation for using machine learning in fluid simulation is the high computational cost of certain solver components. In particular, pressure projection is often the most expensive step, accounting for up to 60–90% of the total runtime in many solvers. Machine learning models are therefore commonly employed to accelerate pressure solves or to reduce the number of iterations required by classical solvers.

In addition, high-resolution turbulence and fine-scale flow details are expensive to simulate directly. Machine learning can learn to model making it especially useful for visual enhancement and real-time applications.

3.3.2 Hybrid Physics–ML Solver Architecture. Most successful approaches adopt a hybrid solver architecture. In a typical hybrid loop, a classical solver performs core simulation steps such as advection, boundary handling, and enforcement of incompressibility constraints. A machine learning module then predicts corrections, such as pressure updates, turbulence forces, or viscosity and stress terms. Finally, a physics-based step enforces divergence-free velocity, mass conservation, and numerical stability.

In this setting, machine learning suggests corrections, while physics-based solvers make the final decisions, ensuring that fundamental physical laws are respected.

3.3.3 Why Hybrid Methods Dominate. Purely data-driven fluid simulation models face several limitations, including poor generalization to unseen scenes, instability over long rollouts, and difficulty enforcing boundary conditions. These issues make purely ML-based solvers unreliable for production use.

Hybrid methods overcome these limitations by combining the strengths of both approaches. Physics-based solvers provide stability, conservation laws, and physical plausibility, while machine learning improves computational speed, visual richness, and scalability. As argued by Thuerey et al., physics-guided learning represents the most viable long-term direction for fluid simulation.

3.3.4 Common Hybrid Techniques. Several hybrid techniques are commonly explored in the literature. Neural surrogate models, often based on convolutional or graph neural networks, approximate components such as pressure fields, or velocity updates and are integrated into classical solver loops. Learned closure models predict turbulent stresses or subgrid forces, which are then applied by traditional solvers. Physics-informed neural networks (PINNs) incorporate physical constraints directly into the loss function by enforcing Navier–Stokes residuals and boundary conditions, although these methods are currently more prevalent in research than in production systems.

Overall, machine learning serves as a complementary tool that enhances traditional fluid solvers rather than replacing them, leading to faster and more visually detailed simulations while preserving stability and interpretability.

This literature survey highlights that modern fluid simulation in computer graphics is largely built upon a set of well-established methods that remain actively used today. Offline simulation pipelines continue to rely on hybrid particle–grid approaches such as FLIP and its extensions to achieve high visual realism, while real-time applications favor particle-based methods that emphasize stability and performance, including Position-Based Fluids and Smoothed Particle Hydrodynamics.

In parallel, recent advances in machine learning have led to hybrid physics–ML approaches that augment classical solvers rather than replacing them. These methods focus on accelerating computationally expensive components and enriching visual detail while preserving physical constraints enforced by traditional simulation techniques. Across all categories, a common trend emerges: practical systems prioritize robustness, controllability, and interpretability over purely data-driven solutions.

Overall, the reviewed methods demonstrate that contemporary fluid simulation relies on a careful balance between physical accuracy, computational efficiency, and application-specific constraints. This survey provides the theoretical and practical context that motivates the design choices made in this project and helps situate the implemented techniques within current research and production practices.

4 TECHNICAL DETAILS

4.1 Geometry Processing

The project implements several geometry processing operations applied to .off meshes. Laplacian smoothing ,Taubin smoothing, curvature weight smoothing and Mesh decimation

A significant limitation arises from the use of OBJ files, which often duplicate vertices per face and lack explicit connectivity information. Geometry processing algorithms relying on vertex adjacency may therefore cause meshes to break, i'll explain that later in the report.

4.2 Laplacian Smoothing

The goal of Laplacian smoothing is to smooth a mesh while keeping its global shape as much as possible. The core idea is to move each vertex toward the average position of its neighboring vertices, thereby reducing local surface irregularities.

Figure 5 shows the effect of Laplacian smoothing applied to the rhinoceros mesh. The original mesh exhibits noticeable high-frequency geometric noise and irregularities due to its dense triangulation. After applying Laplacian smoothing, the surface becomes noticeably smoother, with small-scale noise reduced across the model.

However, this smoothing operation also introduces a well-known side effect: volume shrinkage. The overall shape of the rhinoceros becomes slightly thinner, especially in curved regions such as the torso and head. This observation motivated the later use of Taubin smoothing, which compensates for this shrinkage while retaining the smoothing effect.

For the rest of the geometry processing that i did, the screenshots were unclear, so i did not put pictures about it in this report. But i did put video recordings about the geometrical transformation in my video presentation.

4.2.1 Neighborhood Construction. For each vertex v_i, all its 1-ring neighboring vertices are collected using mesh connectivity information derived from triangle adjacency. This step defines the local geometric neighborhood of the vertex and is essential for computing local surface behavior.

4.2.2 Laplacian Displacement. Once the neighborhood is defined, the Laplacian displacement is computed as:

$$\Delta v_i = \bar{v}_{\text{neighbors}} - v_i,$$

where $\bar{v}_{\text{neighbors}}$ is the average position of the neighboring vertices of v_i.

4.2.3 Vertex Update. Each vertex is then updated according to:

$$v_i \leftarrow v_i + \lambda \Delta v_i,$$

where λ is a user-defined smoothing parameter. Small values of λ result in gentle smoothing, while larger values produce more aggressive smoothing.

While Laplacian smoothing is effective at reducing noise, a known limitation is volume shrinkage. This occurs because each iteration moves vertices inward toward their neighbors, causing the mesh to gradually collapse.

4.3 Taubin Smoothing

To address the volume shrinkage introduced by Laplacian smoothing, Taubin smoothing is applied. The key idea is to combine two Laplacian smoothing steps with opposite weights in order to cancel the shrinkage effect while preserving smoothness.

The algorithm consists of:

- A first Laplacian smoothing step with a positive weight $\lambda > 0$ (smoothing step),
- A second Laplacian smoothing step with a negative weight $\mu < 0$ (inflation step).

By alternating these two steps, the inward motion of vertices caused by the first step is compensated by the second step. This results in a smoother mesh with significantly better shape preservation.

In the project, Taubin smoothing is applied interactively (triggered by a key press) and is particularly useful for fairing organic surfaces such as the rhinoceros mesh, where preserving volume and overall silhouette is important.

4.4 Mesh Decimation

Mesh decimation is used to reduce the number of triangles in a mesh while preserving its overall shape. In this project, decimation is primarily motivated by performance considerations, especially to improve ray tracing efficiency by reducing the number of triangle intersection tests.

Figure 4 illustrates the effect of mesh decimation on the apple model. The first image shows the original apple mesh before any processing, which contains a dense triangle distribution. While visually accurate, this level of geometric detail is unnecessary for distant rendering and severely impacts performance, especially for ray tracing.

After a first decimation pass, the triangle count is significantly reduced while the overall silhouette and visual appearance of the apple remain largely unchanged. A second consecutive decimation further simplifies the mesh, resulting in a much coarser representation. At this stage, some geometric details are visibly lost, but the object still remains recognizable.

These results highlight the trade-off between geometric complexity and performance. In practice, the decimated apple mesh offered a good compromise, allowing faster ray tracing while preserving acceptable visual quality.

4.4.1 Candidate Edge Selection. At each iteration, a candidate edge is selected for collapse. Possible strategies include choosing the shortest edge, selecting a random edge, or using an error-based metric. In this implementation, a simple decimation ratio is used, meaning that edges are removed iteratively until a target reduction ratio is reached, without relying on complex error metrics. This keeps the algorithm simple and fast.

4.4.2 New Vertex Position. When an edge (v_i, v_j) is collapsed, a new vertex position v' must be computed. Common choices include:

$$v' = \frac{v_i + v_j}{2},$$

using the midpoint, or choosing one of the original endpoints. In this project, either the midpoint or one endpoint is used, providing a stable and efficient solution.

4.4.3 Connectivity Update. After collapsing an edge, all faces adjacent to that edge are removed, and neighboring triangles are updated to reference the new vertex. Care is taken to avoid degenerate triangles, flipped normals, and non-manifold edges, ensuring that the resulting mesh remains valid.

4.4.4 Termination. If the initial triangle count is N, the target triangle count is set to $0.5N$. The decimation process is repeated until this target is reached.

4.4.5 Curvature-Weighted Smoothing. In order to better preserve geometric features during smoothing,i implemented a curvature-weighted variant of Laplacian smoothing . Instead of applying a uniform smoothing strength across the surface, the method adapts the displacement of each vertex according to an estimate of local curvature.

For each vertex, a discrete Laplacian vector is computed as the difference between the vertex position and the average of its one-ring neighbors:

$$\Delta \mathbf{x}_i = \bar{\mathbf{x}}_i - \mathbf{x}_i.$$

The magnitude of this vector provides a simple approximation of the local mean curvature. Vertices located in highly curved regions produce larger Laplacian magnitudes, while vertices on flatter areas yield smaller values.

This curvature estimate is normalized and used to compute a weight that scales the smoothing step:

$$\mathbf{x}_i^{new} = \mathbf{x}_i + \lambda\, w_i\, (\bar{\mathbf{x}}_i - \mathbf{x}_i),$$

where w_i is a curvature-dependent weight and λ is a global smoothing parameter. As a result, stronger smoothing is applied in regions of high curvature, while flatter regions are minimally affected. Although the curvature estimation remains approximate, this approach improves feature preservation compared to uniform Laplacian smoothing and proved effective in practice.

5 RIGID BODY SIMULATION

This project includes a simple rigid body simulation illustrating the motion of a falling apple. The simulation is adapted from the rigid body solver initially provided for a cubic dice, and extended to handle a spherical rigid body. The apple is modeled as a rigid sphere governed by Newton's laws of motion, including both linear and angular dynamics, and is subject to gravity, ground collision, friction, and restitution.

5.1 Physical Model

The apple is treated as a rigid body with fixed mass and inertia. Its motion is described by:

- Linear motion, governed by linear momentum and external forces
- Angular motion, governed by angular momentum and applied torques

The rigid body is affected by gravity, collision with the ground plane, friction forces at the contact point, and restitution during impacts.

5.2 Linear Motion

Linear motion is computed using Newton's second law in momentum form. The linear momentum $\mathbf{P}$ is updated according to:

$$\mathbf{P}_{t+1} = \mathbf{P}_t + \Delta t\, \mathbf{F},$$

where $\mathbf{F}$ is the total force applied to the body. The linear velocity is obtained as:

$$\mathbf{V} = \frac{\mathbf{P}}{M},$$

and the position is updated using explicit Euler integration:

$$\mathbf{X}_{t+1} = \mathbf{X}_t + \Delta t\, \mathbf{V}.$$

In the implementation, gravity is applied as a constant force:

$$\mathbf{F}_{\text{gravity}} = M\mathbf{g}.$$

5.3 Angular Motion

Angular motion is handled using angular momentum $\mathbf{L}$. The angular momentum is updated as:

$$\mathbf{L}_{t+1} = \mathbf{L}_t + \Delta t\, \boldsymbol{\tau},$$

where $\boldsymbol{\tau}$ is the applied torque.

The angular velocity is computed using the inverse inertia tensor:

$$\boldsymbol{\omega} = \mathbf{I}^{-1}\mathbf{L}.$$

For the apple, which is modeled as a sphere, the inertia tensor in body space is isotropic:

$$\mathbf{I}_0 = \frac{2}{5} M r^2 \mathbf{I},$$

where r is the sphere radius. The inverse inertia tensor in world space is updated every step as:

$$\mathbf{I}^{-1} = \mathbf{R}\mathbf{I}_0^{-1}\mathbf{R}^T.$$

5.4 Collision with the Floor

Collision handling is implemented specifically for a sphere interacting with a horizontal ground plane. A collision occurs when:

$$X_y - r < y_{\text{floor}}.$$

When a collision is detected:

- The position is corrected so the sphere rests on the floor
- The vertical velocity is inverted and scaled by a restitution coefficient
- Horizontal velocity components are damped to simulate friction

This results in realistic bouncing behavior.

5.5 Friction and Angular Effects

To simulate rolling motion, friction forces are applied at the contact point between the sphere and the ground. The velocity at the contact point is computed as:

$$\mathbf{v}_{\text{contact}} = \mathbf{V} + \boldsymbol{\omega} \times \mathbf{r},$$

where $\mathbf{r}$ is the vector from the center of mass to the contact point.

The tangential component of this velocity is used to compute a friction force, which generates a torque:

$$\boldsymbol{\tau}_{\text{friction}} = \mathbf{r} \times \mathbf{F}_{\text{friction}}.$$

This torque updates the angular momentum and produces realistic rolling after impact.

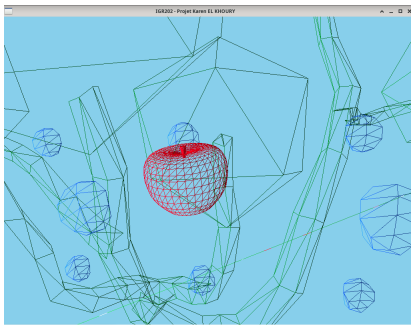

Figure 1: Apple mesh before any processing

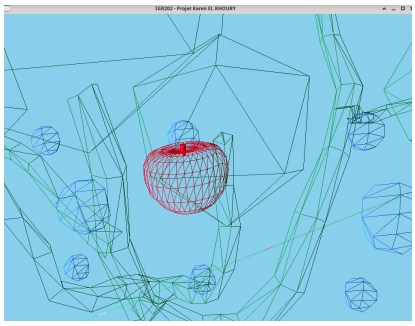

Figure 2: Apple mesh after applying decimation

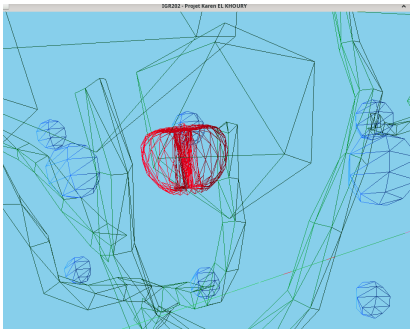

Figure 3: Apple mesh after two consecutive mesh decimations

Fig. 4. Effect of mesh decimation on the apple model.

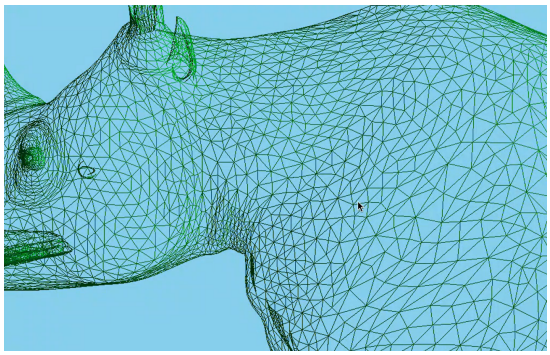

Figure 1: Rhino Mesh before any processing

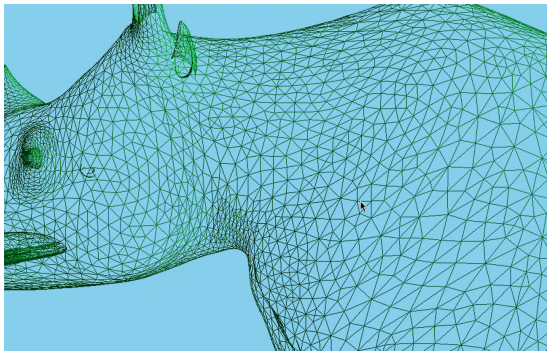

Figure 2: Rhino Mesh After Laplacian Smoothing

Fig. 5. Effect of Laplacian Smoothing on the rhino model.

5.6 Adaptation from the Dice Simulation

The rigid body solver was originally designed to simulate a cubic dice. The core integration scheme for linear and angular motion remains unchanged. However, several adaptations were made for the apple:

- The dice inertia tensor was replaced by the analytical inertia tensor of a sphere
- Collision handling was simplified from face-based box collisions to sphere–plane collision
- Friction and torque computation were adapted to use a single contact point

These changes allow the same rigid body framework to be reused while accurately modeling spherical motion. The result is a physically plausible falling, bouncing, and rolling apple.

5.7 My SPH implementation

6 SMOOTHED PARTICLE HYDRODYNAMICS (SPH)

6.1 Overview

In this project, I implemented a fluid simulation using the Smoothed Particle Hydrodynamics (SPH) method. I represented the fluid as a set of particles storing position, velocity, density, and pressure. All physical interactions are computed locally using a smoothing kernel.

I used the normalization factor $\frac{8}{\pi h^3}$ to ensure correct mass conservation in 3D.

6.2 Density Computation

I computed the density of each particle using:

$$\rho_i = \sum_j m_0 W(\mathbf{x}_i - \mathbf{x}_j)$$

where m_0 is the particle mass and W is the smoothing kernel.

6.3 Pressure Equation

I computed pressure using an equation of state:

$$p_i = k\left(\left(\frac{\rho_i}{\rho_0}\right)^{\gamma} - 1\right)$$

where:

- ρ_0 is the rest density,
- k is the stiffness coefficient,
- γ controls compressibility.

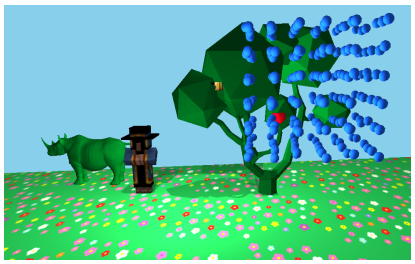

Figure 1: Apple mesh before any processing

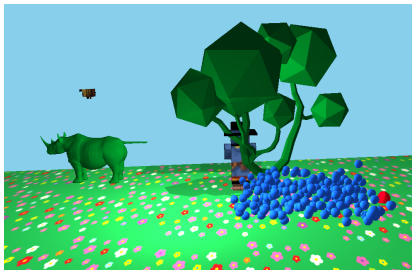

Figure 2: Apple mesh after applying decimation

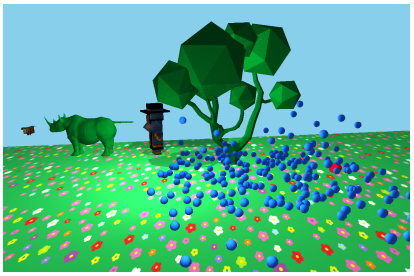

Figure 3: Apple mesh after two consecutive mesh decimations

Fig. 6. Effect of mesh decimation on the apple model.

I used a high stiffness value to make the fluid nearly incompressible.

6.4 Forces

For each particle, I computed acceleration by summing different forces:

Gravity. I added gravity directly:

$$\mathbf{a}_i^{grav} = \mathbf{g}$$

Pressure Force. I implemented the symmetric SPH pressure force:

$$\mathbf{a}_i^{press} = -\sum_j m_0 \left(\frac{p_i}{\rho_i^2} + \frac{p_j}{\rho_j^2} \right) \nabla W(\mathbf{x}_i - \mathbf{x}_j)$$

Viscosity Force. I added viscosity to stabilize the motion:

$$\mathbf{a}_i^{visc} = \sum_j 2\nu m_0 \frac{(\mathbf{v}_j - \mathbf{v}_i) \cdot \mathbf{r}_{ij}}{\rho_j (\|\mathbf{r}_{ij}\|^2 + \epsilon h^2)} \nabla W(\mathbf{r}_{ij})$$

This smooths velocity differences between neighboring particles.

6.5 Spatial Acceleration Structure

To reduce computational complexity, I implemented a uniform 3D grid. Since the kernel support radius is $2h$, I searched only within neighboring cells. This reduces complexity from $O(N^2)$ to approximately $O(N)$.

6.6 Numerical Stability and Parameter Tuning

One of the main challenges I encountered during the implementation was numerical instability. At the beginning, the simulation was frequently exploding, with particles accelerating uncontrollably and leaving the domain. This behavior was mainly caused by large pressure forces due to inappropriate parameter combinations.

The SPH method is highly sensitive to the choice of parameters such as the time step Δt, stiffness coefficient k, rest density ρ_0, viscosity ν, smoothing length h, and particle mass m_0. Small changes in these values can drastically affect stability.

In particular:

- If the time step was too large, the integration became unstable.
- If the stiffness k was too high, pressure forces became excessively strong and caused numerical explosions.
- If viscosity was too low, particles exhibited chaotic oscillations.

Finding a stable configuration required extensive experimentation. I progressively reduced the time step, increased viscosity, and adjusted the stiffness coefficient until I obtained a stable, nearly incompressible fluid behavior.

This tuning process was essential to prevent particle overlap, unrealistic accelerations, and energy blow-up.

6.7 Rendering

6.8 Shadow Mapping and Interactive Lighting

Real-time shadows are implemented using shadow mapping, a two-pass rendering algorithm. In the first pass, the scene is rendered from the point of view of each light source, and the resulting depth values are stored in a shadow map texture. In the second pass, during the main render, each fragment is transformed into the corresponding light space and its depth is compared against the stored shadow map. If the fragment lies behind the stored depth, it is classified as being in shadow. This approach allows the generation of dynamic shadows in real time and supports multiple light sources.

In addition to shadow mapping, the scene includes interactive and animated lighting effects. Each light source is defined by its position, color, and intensity. In a disco mode, light colors and intensities vary over time using sinusoidal functions, creating animated and visually dynamic lighting. This results in dancing lights that enhance the liveliness of the scene while remaining fully compatible with the shadow mapping pipeline.

7 RAY TRACING

Figure 7 illustrates the progressive development of the ray tracing pipeline, from early debugging visualizations to the final shaded and shadowed results.

The first image visualizes surface normals encoded as RGB colors. This representation was used as an initial debugging step to validate the correctness of ray–triangle intersections and normal computation. Each visible color corresponds to a normalized surface normal direction, allowing geometric errors and orientation issues to be easily identified.

The second image shows an intermediate ray-traced frame rendered using simple diffuse shading without texture mapping. At

this stage, the lighting model is functional, but surface appearance remains uniform. This step confirms correct light direction handling and visibility computation before introducing texture sampling.

The third image presents the scene after texture mapping is added but before hard shadow computation. While the scene gains visual richness through textured surfaces, the lack of shadows results in a flat appearance and reduced depth perception.

7.1 Ray Generation

The ray tracer follows a pinhole camera model and generates one primary ray per pixel. For a pixel with coordinates (x, y), a ray is defined as:

$$\mathbf{r}(t) = \mathbf{o} + t\mathbf{d},$$

where $\mathbf{o}$ is the camera position and $\mathbf{d}$ is the ray direction.

Pixel coordinates are first converted into view space:

$$u = \frac{x - W/2}{W}, \quad v = \frac{y - H/2}{H},$$

and the ray direction is computed as:

$$\mathbf{d} = \text{normalize}(\langle u, -v, -1 \rangle).$$

This shoots rays through an image plane located in front of the camera.

7.2 Scene Traversal

For each generated ray, the scene is traversed using a brute-force approach. The ray is tested against all meshes in the scene, and against all triangles in each mesh. This results in a time complexity of $O(N_{\text{triangles}})$ per ray.

Although slow, this design choice is intentional. The brute-force traversal keeps the algorithm simple, explicit, and easy to understand, which is appropriate for an educational ray tracer.

7.3 Ray–Triangle Intersection

Ray–triangle intersection is computed using the Möller–Trumbore algorithm, which is the core of the ray tracer. A triangle is defined by its vertices (v_0, v_1, v_2), with edges:

$$\mathbf{e}_1 = v_1 - v_0, \quad \mathbf{e}_2 = v_2 - v_0.$$

The intersection test solves:

$$\mathbf{o} + t\mathbf{d} = v_0 + u\mathbf{e}_1 + v\mathbf{e}_2,$$

where t is the distance along the ray and (u, v) are barycentric coordinates.

An intersection is valid if:

$$u \geq 0, \quad v \geq 0, \quad u + v \leq 1, \quad t > 0.$$

7.4 Closest-Hit Selection

For each ray, the intersection with the smallest positive t value is retained. This closest-hit logic ensures that only the nearest visible surface contributes to the final pixel color. All other intersections along the ray are ignored.

7.5 Surface Data at Intersection

Once a valid intersection is found, surface properties are computed. The surface normal is obtained from the triangle geometry:

$$\mathbf{n} = \text{normalize}\left((v_1 - v_0) \times (v_2 - v_0)\right).$$

Flat shading is used, which is sufficient for Lambertian lighting.

Texture coordinates are interpolated using barycentric coordinates:

$$\text{UV} = (1 - u - v)\,\text{UV}_0 + u\,\text{UV}_1 + v\,\text{UV}_2.$$

This interpolation ensures correct texture lookup without distortion.

7.6 Shading Model

The ray tracer uses a Lambertian shading model to simulate diffuse reflection. The lighting term is computed as:

$$I = \max(\mathbf{n} \cdot \mathbf{l}, 0),$$

where $\mathbf{n}$ is the surface normal and $\mathbf{l}$ is the light direction.

The final color is obtained by combining the diffuse term with a texture-based albedo:

$$\mathbf{c} = \mathbf{c}_{\text{texture}} \cdot I.$$

In this model, geometry determines visibility while the texture defines surface color.

7.7 Shadow Rays

Hard shadows are computed using shadow rays. From each intersection point $\mathbf{p}$, a secondary ray is cast toward the light source:

$$\mathbf{r}_{\text{shadow}}(t) = \mathbf{p} + \epsilon\mathbf{n} + t\mathbf{l},$$

where a small offset ϵ is applied to avoid self-shadowing artifacts.

If an intersection occurs before reaching the light, the point is considered in shadow and its shading is darkened. If no intersection is found, full diffuse lighting is applied. If no primary intersection exists, a constant sky color is returned.

This approach produces hard shadows and clearly illustrates the role of secondary rays in ray tracing.

Figure 8 shows three frames extracted from the final ray-traced animation. These frames demonstrate the complete ray tracing pipeline, including texture mapping and hard shadows computed using secondary shadow rays. The apple can be seen falling over time, with its motion reflected consistently in the ray-traced output. As i will explain later, i did raytracing without the rhino mesh because it contains too many triangles and takes too much time for a frame to be rendered.

Together, these images highlight the incremental construction of the ray tracer and emphasize the importance of validating individual components—such as normals, shading, texturing, and shadow rays—before combining them into a full rendering pipeline.

8 MODIFICATIONS AND IMPROVEMENTS

During the development of the project, several modifications and practical improvements were introduced to adapt existing algorithms to the target scene and to improve robustness, performance, and visual quality.

Figure 1: Ray tracing and representing the normals by colors

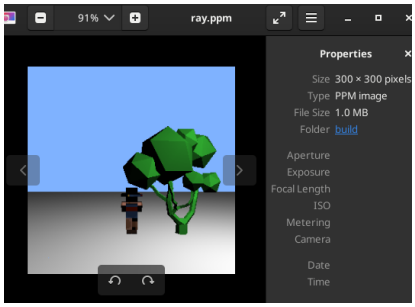

Figure 2: Raytracing frame before adding texture

Figure 3: Raytracing without hard Shadows

Fig. 7. Evolution of my raytraced scene

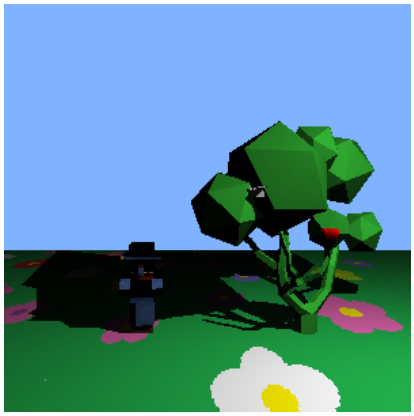

Figure 1: Frame 1

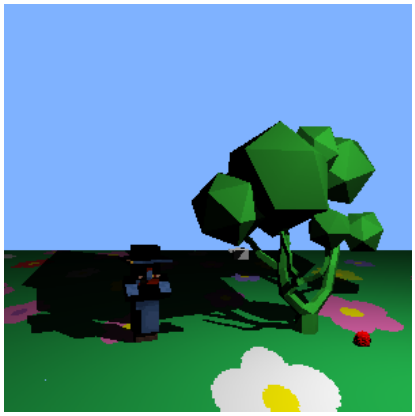

Figure 2: Frame 6

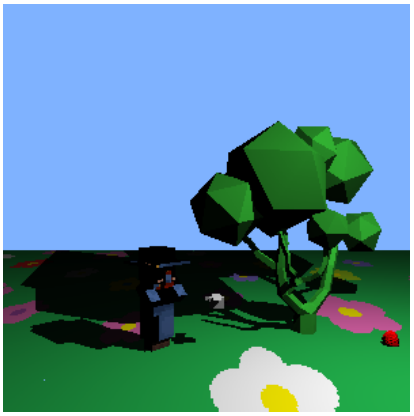

Figure 3: Frame 9

Fig. 8. Three consecutive frames from the ray-traced animation showing the apple motion over time.

8.1 Rigid Body Solver Adaptation

The rigid body solver was initially designed to simulate a cubic dice (TP RIGID BODY SIMULATION). Rather than rewriting the solver from scratch, i adapted to simulate a falling apple represented as a rigid sphere. This required replacing the box inertia tensor with the analytical inertia tensor of a sphere and simplifying collision handling from box–plane interactions to sphere–plane collisions. Friction forces applied at the contact point were used to generate torque, producing realistic rolling motion after impact. This modification demonstrates how a generic rigid body framework can be reused and adapted to different shapes.

8.2 Improved Mesh Smoothing

Experiments with Laplacian smoothing revealed significant volume shrinkage, especially on organic meshes. To address this issue, Taubin smoothing was introduced by combining two Laplacian steps with positive and negative weights. This modification preserves the overall shape.

8.3 Ray Tracing Design

The ray tracer was implemented using a straightforward brute-force traversal strategy, where each ray is tested against all triangles in the scene. In practice, this design choice quickly revealed severe performance limitations. Even at extremely low image resolutions, rendering a single frame required a very large amount of time when complex geometry was present in the scene .

This issue was particularly evident when ray tracing the rhinoceros model, which contains a very high number of triangles. The cost of testing every ray against this geometry made rendering prohibitively slow and prevented effective debugging and validation of the ray tracing pipeline. As a result, it became difficult to iterate on the implementation or visually inspect intermediate results.

To address this limitation, the rhinoceros model was removed from the ray-traced scene, allowing rendering times to become manageable. This simplification made it possible to increase image resolution and clearly observe shading, textures, and lighting effects on simpler objects such as the apple model. In order to generate animations, ray tracing was performed at discrete time interval. Individual frames were rendered sequentially and later assembled into a short animation by placing the frames side by side.

This approach enabled the ray tracer to be used effectively despite its computational cost. While the underlying algorithm remains inefficient, the adopted workflow made it possible to validate correctness, visualize results, and produce meaningful animations. The implementation therefore serves as a clear and explicit baseline for understanding ray tracing, while highlighting the practical challenges that motivate more advanced acceleration techniques.

8.4 Mesh Decimation for Performance

As mentioned, Ray tracing performance is directly affected by the number of triangles in the scene. To reduce computational cost, a simple mesh decimation strategy based on edge collapse was implemented. The triangle count of the apple mesh was reduced by approximately 50%, which significantly improved ray tracing performance while maintaining visual quality. Despite this optimization, the rhinoceros mesh remained too complex and was therefore removed from the ray-traced scene, illustrating the trade-off between geometric detail and rendering performance, but i was able to keep the apple mesh after decimation and ray tracing didn't take much time (around 30 seconds).

9 OBSERVATIONS AND FUTURE WORK

This project highlighted several practical limitations related to ray tracing performance and geometry processing. These observations directly influenced design choices made during development and motivate several possible improvements.

9.1 Limitations of the Ray Tracer

The main limitation of the implemented ray tracer is performance. Rendering even small images takes several seconds, making interactive use impossible. This is expected given the algorithmic structure of the ray tracer.

For each pixel, a primary ray is generated. For each ray, intersection tests are performed against all meshes in the scene and against all triangles within each mesh. As a result, the overall complexity is approximately:

$$O(N_{\text{pixels}} \times N_{\text{triangles}}).$$

Shadow rays follow the same brute-force process, further increasing computational cost. The ray tracer performs no spatial filtering or acceleration, meaning that every ray tests every triangle.

Due to this performance limitation, the rhinoceros mesh had to be removed from the ray-traced scene, as it contained a very large number of triangles and made rendering prohibitively slow. In contrast, the apple mesh was retained by applying mesh decimation to reduce its triangle count, which significantly improved ray tracing performance while preserving its overall shape.

9.2 Lack of Acceleration Structures

A key missing component in the ray tracer is a spatial acceleration structure. Currently, every ray checks every triangle, which is inefficient. Common acceleration structures such as Axis-Aligned Bounding Boxes (AABB), Bounding Volume Hierarchies (BVH), KD-trees, or uniform grids are not implemented.

A possible improvement is to group triangles inside bounding volumes and first test ray–box intersections. Only if a ray intersects a bounding volume would the triangles inside be tested. This would drastically reduce the number of ray–triangle intersection tests. Even a simple approach, such as one AABB per mesh or per group of triangles, would significantly accelerate both primary rays and shadow rays.

9.3 Limitations of OBJ Geometry

Another important limitation observed during the project is related to loading and processing OBJ meshes. When applying geometry processing algorithms such as Laplacian smoothing, Taubin smoothing, or mesh decimation, some OBJ meshes exhibited unexpected behavior. In particular, certain meshes appeared to break apart, with triangles becoming independent and losing surface coherence.

This behavior is explained by limitations of the OBJ format. OBJ is primarily a rendering-oriented format and does not guarantee explicit vertex adjacency or topological consistency. In many OBJ files, vertices are duplicated per face, meaning that two triangles sharing an edge may not share vertex indices. From a geometry processing perspective, such meshes behave as collections of disconnected triangles rather than as a coherent surface.

9.4 Geometry Processing Constraints

Geometry processing algorithms rely on well-defined vertex neighborhoods, shared vertices across faces, and manifold connectivity. When these conditions are not met, algorithms such as smoothing and decimation fail, causing triangles to move independently and the mesh to visually "explode" or float apart.

9.5 Future Work

Several improvements could be made to address these limitations:

- Introduce spatial acceleration structures such as AABB or BVH to reduce ray tracing complexity.
- Preprocess OBJ meshes by merging duplicated vertices and rebuilding vertex adjacency.
- Convert meshes to geometry-friendly formats such as OFF before processing.
- Use half-edge or winged-edge data structures to support robust geometry algorithms.
- Validate mesh manifoldness before applying smoothing or decimation.

10 CONCLUSION

I really enjoyed working on this project throughout the semester, as it provided a valuable opportunity to explore both the theoretical foundations and practical challenges of computer graphics. Implementing geometry processing, rigid body simulation, fluid simulation, and rendering techniques.

My literature survey made me learn a lot and it was really interesting to highlighted the gap between idealized algorithms and real-world constraints. Performance limitations, numerical stability issues, and data representation choices all played a significant role in shaping the final implementation.

Overall, the project was both challenging and rewarding. I liked the idea of doing whatever we wanted in this project.

REFERENCES

[1] J. Stam. 1999. Stable fluids. In *Proceedings of the 26th annual conference on Computer graphics and interactive techniques* (SIGGRAPH '99). ACM, 121–128.

[2] R. Bridson. 2015. *Fluid Simulation for Computer Graphics* (2nd ed.). CRC Press.

[3] Y. Zhu and R. Bridson. 2005. Animating sand as a fluid. *ACM Transactions on Graphics (TOG)* 24, 3 (2005), 965–972.

[4] C. Jiang, C. Schroeder, A. Selle, J. Teran, and A. Stomakhin. 2015. The affine particle-in-cell method. *ACM Transactions on Graphics (TOG)* 34, 4 (2015), 1–11.

[5] M. Macklin and M. Müller. 2013. Position based fluids. *ACM Transactions on Graphics (TOG)* 32, 4 (2013), 1–12.

[6] M. Ihmsen, J. Cornelis, B. Solenthaler, C. Horvath, and M. Teschner. 2014. Implicit Incompressible SPH. *IEEE Transactions on Visualization and Computer Graphics* 20, 3 (2014), 426–435.

[7] N. Thuerey, P. Holl, M. Müller, P. Schnell, F. Tuerko, and K. Um. 2021. *Physics-based Deep Learning*. Springer. https://physicsbaseddeeplearning.org

[8] Blender Foundation. 2024. *Mantaflow Fluid Simulation*. Retrieved from https://docs.blender.org/manual/en/latest/physics/fluid/